Locating and Removing Backdoors in a Time-Series Forecaster

Anonymous submission

Abstract

A backdoor planted through a forecaster's training data makes the model emit an attacker's pattern whenever a short trigger appears in its input. We study where such a backdoor sits in PatchTST and whether it can be removed without a clean reference model. Each poisoned model is trained next to a clean twin with the same initialization, seed and data order. For the random trigger, reverting all attention or all MLP to the twin removes most of the backdoor and reverting both removes all of it, while installing either one alone in the twin does not create it, so the backdoor depends on both. The learned trigger leans more on attention. We then adapt an immunization-style purification that builds its own backdoored variants from a small clean set and suppresses the units that change most and most consistently. With 30% of the units in each module suppressed, 2 of 17 implanted random-trigger models and 2 of 16 learned-trigger models are left implanted, against 8 of 17 and 9 of 16 for random units, and 61 to 85% of the gap to a clean model is closed at a clean-MAE cost of about 3%. Purification beats fine-tuning and fine-pruning on all but one model, and its advantage over random units has a paired bootstrap interval above zero only for the learned trigger. A clean set cut from the poisoned data largely removes the purification's effect.

Introduction

Many forecasters are trained on data their owners do not fully control, so a poisoned training set is a realistic way to plant a backdoor (Lin et al. 2024; Xiang et al. 2025; Liu et al. 2026). The poisoned model forecasts normally until a short trigger shows up in the input window, and then it predicts the pattern the attacker chose. Most existing work focuses on concealing the trigger, while leaving the internal location of the backdoor largely unexplored. Understanding how the backdoor is encoded in the model's weights could help defenders identify and remove it.

We study this in PatchTST (Nie et al. 2023). We compare each poisoned model with a clean twin and measure how much of the backdoor survives when parts of the model are reverted to the twin's weights, and whether installing those parts in the twin creates it. We give an operational criterion for when an attack counts as implanted. It matters here because the attacked channel pair and the input length decide whether a backdoor implants in every seed or in none. We adapt a purification method from LLM backdoor removal (Li and Kim 2026) that needs no clean twin and no knowledge of the trigger values, channels or timestamps, and we compare it with random units, fine-tuning and fine-pruning (Liu, Dolan-Gavitt, and Garg 2018). We also test whether the purification selects the units that carry the backdoor and how it depends on the defender's clean set.

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

Related Work

Backdoors in forecasting. BackTime (Lin et al. 2024) poisons a small share of timestamps with a learned trigger and a fixed target pattern. BadTime (Xiang et al. 2025) and TDBA (Liu et al. 2026) add other triggers and delayed activation, and Chen et al. (2026) tailor the attack to forecast-driven grid operations. We keep BackTime's data-poisoning threat model, its pooled scaling of trigger and target, and its two metrics, the clean MAE and the MAE against the target pattern. Related attacks on time series classifiers (Jiang et al. 2022; Huang et al. 2025; Dong et al. 2025) aim to induce incorrect labels rather than manipulate forecast outputs.

Defenses on time series. TimeGuard (Nguyen et al. 2026) is a training-time defense for forecasting. It starts from the observation that sample-filtering and trigger-synthesis defenses fail to localize backdoors in forecasters, and it assumes the defender controls the training run. Anti-backdoor learning (Jiang et al. 2024) also acts during training, for classification. Unlike TimeGuard, our method operates on a trained model, given only its weights and a small clean set. The Trojan Horse Hunt competition (Kotowski et al. 2026) challenges participants to identify triggers in forecasting models. Our method does not require prior knowledge of the trigger.

Backdoor removal and localization. Fine-pruning (Liu, Dolan-Gavitt, and Garg 2018) and adversarial neuron pruning (Wu and Wang 2021) remove neurons from image classifiers. In language models, Lamparth and Reuel (2024) find backdoors in early MLP layers and embeddings, and Li and Kim (2026) report that MLPs hold the trigger-behavior association while attention mostly amplifies the trigger. The latter also proposes purification without a clean reference model. It generates backdoored variants of the suspect model and

uses their weight differences to score its components. We adapt their scoring method to forecasting models, evaluating MLP neurons and attention heads as potential locations of the backdoor. In PatchTST, attention matters about as much as the MLP (Figure 1), although reverting to a twin and zeroing a module are different interventions, so the two findings are not directly comparable.

Threat Model

The attacker poisons a small share of the training timestamps. At each poisoned timestamp, it inserts a short trigger into a few channels and places a target pattern immediately after it in the same channels. A model trained on this data learns to produce the target pattern whenever the trigger appears. The attacker does not modify the training procedure or architecture, following BackTime's data-poisoning setting (Lin et al. 2024). The defender receives the trained weights and architecture, along with a small clean set from a trusted source, such as records from the same system. It does not know whether the model is poisoned, the trigger values, the attacked channels, or the poisoned timestamps. To construct its variants, it uses BackTime's attack template, which is also used by the attacks evaluated here: a cone-shaped target pattern of length 7, a trigger length of 4, and an amplitude of 0.2 in pooled units. Trigger values, channels, and timestamps are sampled at random. Our localization study is the only exception. It uses a clean twin unavailable to a real defender, solely to demonstrate where the backdoor is encoded in the model.

Setup

Models and data. PatchTST has three layers, width 128, 16 attention heads, patch length 4, and stride 2. The input and forecast lengths are both 12, and the ETTm1 dataset has seven channels. We use seeds 200–209 throughout the experiments, except for the validation sweep, which uses seeds 200 and 201. Each model corresponds to one seed and one trigger pair. This gives 20 models per trigger on the easy and medium pairs. On the hard pair, the random trigger adds 10 models, none of which successfully implant. The localization and purification analyses use the 17 random-trigger and 16 learned-trigger models that implant at input length 12. Each poisoned model has a clean twin trained from the same initialization and data order. We also evaluate a secondary input length of 64 steps.

Attack. The main attack uses a fixed random trigger of length 4, added to the last values of two channels, followed by BackTime's cone-shaped target pattern of length 7. As in BackTime, the trigger amplitude ($\epsilon = 0.2$) and target pattern are scaled using one pooled standard deviation across all channels. We poison 3% of the timestamps. We keep the attacked channel pair fixed across seeds because the choice of channels strongly affects the outcome. We evaluate three pairs: LUFL and LULL (easy), HULL and OT (medium), and HUFL and MUFL (hard). We also evaluate BackTime's learned-trigger attack, in which a trigger generator is trained against a surrogate forecaster. We use the same poisoning budget, scaling, and channel pairs. The generator and surrogate are trained for 100 epochs, with a 10-epoch warm-up and a 12-step surrogate input window. The learned trigger is inserted at the same 3% of timestamps but is not evaluated on the hard pair.

Metrics and the implant criterion. MAE_C is the clean test MAE. For a model m, MAE_P measures how close its forecast is to the attacker's target on triggered windows:

$$\mathrm{MAE}_P(m) = \frac{1}{7\,|V|\,N} \sum_{n=1}^{N} \sum_{t=1}^{7} \sum_{c \in V} \left| \hat{y}_{t,c}^{(n)} - \left(b_c^{(n)} + \pi_{t,c} \right) \right|. \tag{1}$$

Here N is the number of test windows, each with the trigger written into its last four input steps, V is the set of attacked channels, $\hat{y}$ is the forecast of m, b is the last input value before the trigger and π is the target pattern. Everything is in standardized units, and lower means a stronger backdoor. Two references give MAE_P a scale. The flat forecaster predicts $\hat{y}_{t,c} = b_c$ and ignores the trigger. The clean twin of m is trained without the attack, so it shows what the trigger alone does to a model that was never poisoned. Writing P for MAE_P, the ratio $\rho(m) = P(m) \,/\, \min(P_{\mathrm{twin}}, P_{\mathrm{flat}})$ says how far m is below the better of the two. We call a model implanted if $\rho \le 0.5$, partly implanted if $0.5 < \rho < 1$, and not implanted if $\rho \ge 1$. Partly implanted models reproduce some of the target pattern but do not meet the implantation criterion, so we also report ρ alongside the counts. The threshold of 0.5 requires the model to reduce the error of the better reference by at least half. Table 2 reports how the implantation counts change under other thresholds.

Training details. PatchTST is trained for 30 epochs with AdamW (learning rate 10^{-4}, 5% warm-up, and cosine decay to 0.1 times the initial learning rate), an effective batch size of 128, dropout of 0.2, and instance normalization. We use ETTm1 (Zhou et al. 2021), a standard time-series forecasting benchmark also used by BackTime. We follow the standard train, validation, and test split and standardize the data using training statistics. For each seed, the poisoned model and its clean twin share the initialization, data order, and dropout masks. Evaluation is performed in 32-bit precision. The defender's variants are fine-tuned for 300 steps at a learning rate of 5×10^{-5}. Recovery uses 300 steps at 10^{-5}. The defender's trigger has an amplitude of 0.2 in pooled units and poisons 10% of timestamps in the clean set, with $\lambda = 0.01$.

When a Backdoor Implants

Table 1 shows that successful implantation depends strongly on the input length, forecast horizon, and attacked channels. The random trigger implants reliably only with a short input and horizon and the low-variance channels. The learned trigger improves implantation mainly in settings where the random trigger struggles. On the easy pair with the longer input it implants in every seed, against 6 of 10 for the random trigger. On the medium pair it is no stronger (6 of 10 seeds against 7 of 10 at input 12), and the clean twin already follows part of it, with an error on the target pattern of about half the flat reference, which makes the implantation

Pair	Input	random trigger		learned trigger	
		Impl.	ρ	Impl.	ρ
easy	12	10/10	0.07	10/10	0.08
medium	12	7/10	0.47	6/10	0.47
hard	12	0/10	1.04	n/a	n/a
easy	64	6/10	0.53	10/10	0.20
medium	64	0/10	1.04	0/10	0.76
hard	64	0/10	1.18	n/a	n/a

Table 1: PatchTST on ETTm1 (horizon 12), seeds 200 to 209 for both triggers. Impl. counts implanted seeds, and ρ is the seed mean (implanted at 0.5 or below). The learned trigger was not run on the hard pair.

Trigger	Pair	0.25	0.35	0.5	0.75	1.0
random	easy	10/10	10/10	10/10	10/10	10/10
random	medium	1/10	3/10	7/10	8/10	10/10
learned	easy	10/10	10/10	10/10	10/10	10/10
learned	medium	0/10	2/10	6/10	10/10	10/10

Table 2: Seeds implanted at input 12 when the factor of the criterion is varied, for seeds 200 to 209.

criterion harder to satisfy on this pair. Pooled scaling gives every channel the same raw perturbation, so a low-variance channel gets a large trigger in standardized units. This may explain why these channel pairs are easier to implant. With 4 or 8 attention heads in place of 16, the easy pair still implants in 10 of 10 seeds (ρ 0.08 and 0.07) and the medium pair in 5 and 6 of 10 (0.53 and 0.50). With input and horizon set to 24, 48 and 96 the easy pair implants in 9, 5 and 2 of 10 seeds (ρ 0.19, 0.58 and 0.89), and the medium pair in none (1.04, 1.13 and 1.11).

Dependence on the threshold. Table 2 reports the number of implanted seeds at thresholds from 0.25 to 1. The count on the easy pair is unchanged across thresholds. On the medium pair, the random trigger count increases from 3 of 10 at 0.35 to 8 of 10 at 0.75, while the learned trigger count increases from 2 of 10 to 10 of 10. This indicates that many models on the medium pair fall near the implantation threshold. We retain the 0.5 threshold throughout.

Raw units. The implantation results are consistent in the original units. Table 10 in the appendix reports MAE_P in raw units, following BackTime. On the easy pair, the poisoned models reach an MAE_P of 0.10, compared with 1.55 for the twin and 1.59 for the flat forecaster under the random trigger. The ratio ρ computed in raw units differs from the standardized version by at most 0.02 across all four settings. The remaining analyses use only models that meet the implantation criterion.

Where the Backdoor Lives

For each implanted model we replace chosen parameters with the clean twin's values and measure the retained share of the backdoor, $(P_{\text{twin}} - P_{\text{abl}})/(P_{\text{twin}} - P_{\text{poisoned}})$. A value of 1

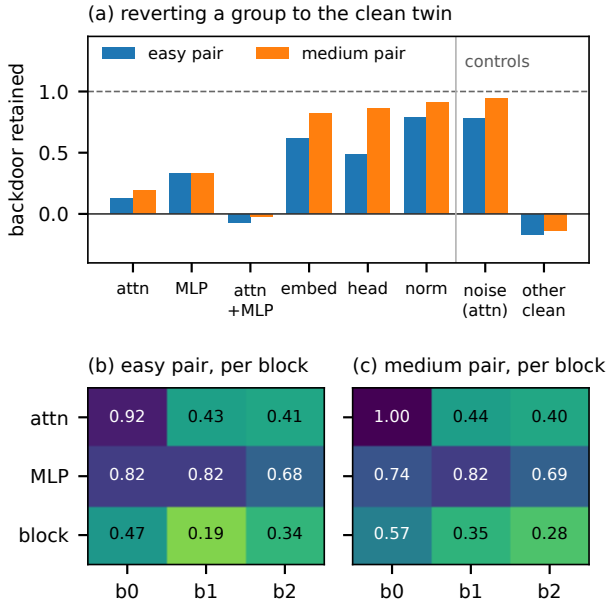


Figure 1: Backdoor retention after replacing model components with their clean-twin weights (1 = fully retained, 0 = reduced to the clean twin). Results use a 12-step input window and implanted models only (10 easy, 7 medium). (a) Whole model components and two controls: matched noise and a different clean model. (b, c) Individual components within blocks b0–b2: attention, MLP, or the entire block, for the easy and medium pairs, respectively.

means the backdoor is intact and 0 means the model is as clean as the twin. A value below 0 means the reverted model is farther from the target pattern than the twin is, because of noise or the damage from mixing weights. Reverting all attention or all MLP removes most of the backdoor, and reverting both removes all of it. Reverting the embedding or output head also reduces the backdoor, but neither has as large an effect (Figure 1 and Table 12 in the appendix). Within the encoder, no individual component accounts for the entire backdoor. Reverting one MLP leaves 0.68 to 0.82 of the backdoor, while reverting block-0 attention leaves 0.92 to 1.00. Reverting an entire block leaves 0.47, 0.19, and 0.34 of the backdoor on the easy pair, and 0.57, 0.35, and 0.28 on the medium pair, for blocks 0, 1, and 2, respectively. Thus, block 0 matters least on both pairs. These results suggest that the backdoor depends on interactions between attention and MLP components, particularly in later blocks. Since reverting either all attention or all MLP removes most of it, this suggests that the backdoor requires both attention and MLP components rather than being stored independently in either. Early attention and the embedding have smaller effects. These ablations show which poisoned components the backdoor depends on, but not where it is stored.

For the learned trigger, the easy pair (10 models) shows a similar pattern, although attention has a larger effect. Reverting all attention leaves 0.15 of the backdoor, while reverting

Installed in the twin	random trigger easy	random trigger medium	learned trigger easy	learned trigger medium
attention	0.06	0.17	−0.01	−0.34
MLP	0.05	0.14	0.09	0.13
attention and MLP	0.20	0.63	0.09	0.37
whole encoder	0.33	0.70	0.34	0.71
embedding	−0.13	−0.02	−0.32	−0.49
head	0.02	0.03	−0.06	0.14
normalization	0.04	−0.04	0.16	0.27

Table 3: Share of the backdoor installed when poisoned parts replace the same parts of the clean twin (input length 12, seeds 200 to 209, implanted models only: 10 easy and 7 medium for the random trigger, 10 easy and 6 medium for the learned trigger). The whole encoder is attention, MLP and normalization of all blocks.

all MLP leaves 0.55. Block 1 matters most, with its reversion leaving -0.07. On the medium pair (6 models), MLP has a larger effect than attention. Reverting all attention leaves 0.39, while reverting all MLP leaves 0.17. Reverting both still leaves 0.16, suggesting that some of the backdoor remains in other components. Reverting the normalization layers leaves 0.35. Two controls check whether these effects are simply caused by changing the weights. Adding noise with the same norm as the weight difference leaves 0.78 to 0.99 of the backdoor intact. Reverting to a different clean model raises clean MAE by 13 to 42%, but does not produce the same backdoor reduction. This analysis requires access to the clean twin, making it an oracle study rather than a practical defense.

Sufficiency. The sufficiency test starts from the clean twin and replaces selected components with the corresponding poisoned weights. Table 3 reports the installed share, $(P_{\text{twin}} - P_{\text{hybrid}})/(P_{\text{twin}} - P_{\text{poisoned}})$, where 1 means that the hybrid achieves the same target-pattern error as the poisoned model. Transplanting the poisoned attention, MLP, embedding, head, or normalization layers individually does not reproduce much of the backdoor, with no single component group installing more than 0.27. Transplanting attention and MLP together installs 0.09 to 0.63, while transplanting the whole encoder installs 0.33 to 0.71. The larger values occur on the medium pair, where the whole encoder reproduces a greater share of the backdoor than attention and MLP together. The necessity and sufficiency results both suggest that the backdoor involves attention and MLP components across multiple blocks. However, the hybrids combine weights from independent training runs, so some of the backdoor may be lost because the transplanted components do not fit together with the remaining clean weights. The installed shares may therefore underestimate how much of the backdoor these components could reproduce in a compatible model.

Purification Without a Clean Twin

The defender has the suspect weights, the architecture, and a small clean set (by default, 5% of the training rows sampled before poisoning. A set from the validation split is also tested below). It has no clean twin and does not know the model's trigger values, attacked channels, or poisoned timestamps.

Algorithm 1 Purification without a clean twin

Input: suspect weights θ_s, clean set D, variants N, budget r
for $i = 1$ to N **do**
  draw a random trigger (template pattern), poison a copy D_i of D
  $\theta_i^{bd} \leftarrow$ fine-tune θ_s on D_i
  $\theta_i^{cl} \leftarrow$ fine-tune θ_s on D (same seed and steps)
  $\Delta_i \leftarrow \theta_i^{bd} - \theta_i^{cl}$
end for
for each unit j (MLP neuron or attention head) **do**
  $\text{score}_j \leftarrow \frac{1}{N}\sum_i \|\Delta_{i,j}\| + \lambda\,\text{align}_j$
end for
zero the top $r\%$ of units in every module
fine-tune briefly on D and return the weights

Method. The defender generates $N = 6$ random triggers of its own (random values, channels and timestamps, with the attack template's pattern, lengths and amplitude) and creates a poisoned variant of the clean set for each one. The suspect model is fine-tuned on each poisoned variant and on the unmodified clean set, using the same seed and number of steps. The difference between the two sets of weights is Δ_i. Each MLP neuron and attention head receives a score: $\frac{1}{N}\sum_i \|\Delta_{i,j}\| + \lambda \cdot \text{align}_j$, where align_j is the mean positive cosine similarity between variants (Li and Kim 2026). The top $r\%$ of units in each module (the MLP or the attention of one layer) are set to zero, and a short fine-tune on the clean set follows (Algorithm 1).

Baselines. All baselines use the same clean set and the same fine-tuning budget. These are fine-tuning alone, suppression of the same number of randomly selected units, and fine-pruning (Liu, Dolan-Gavitt, and Garg 2018), which suppresses only MLP neurons and therefore suppresses fewer units than the other two at the same r.

Setting. We chose the setting on the validation split using two seeds. The rule was the smallest budget in the grid (5, 10, 20, 30 and 50%) that leaves zero implanted models on either pair and limits the clean-MAE increase to at most 3% for both poisoned and clean models. This gives $r = 30\%$ with the default settings. Applying this rule requires each model's actual trigger and clean twin, information unavailable to the defender. In addition, the two validation seeds are reused for testing, although the data split is different. Test results for every budget are shown in Figure 2 and Tables 8 and 9.

Results. We report the share of the gap to the reference closed, $(P_{\text{after}} - P_{\text{poisoned}})/(\text{ref} - P_{\text{poisoned}})$, where $\text{ref} = \min(P_{\text{twin}}, P_{\text{flat}})$. A value of 100% means the post-purification error matches the reference error. We also count how many initially implanted models remain implanted after purification. At $r = 30\%$, purification leaves 2 of 10 models implanted on the easy pair and 0 of 7 on the medium pair, against 6 of 10 and 2 of 7 for random units, 10 of 10 and 3 of 7 for fine-pruning, and 10 of 10 and 4 of 7 for fine-tuning alone

Pair	Method	Gap	Left	Cost (twin)
easy	fine-tune only	$9 \pm 7\%$	10/10	+2.4% (+3.7%)
easy	fine-pruning	$17 \pm 10\%$	10/10	+2.4% (+3.7%)
easy	random units	$51 \pm 28\%$	6/10	+2.8% (+3.6%)
easy	purification	$61 \pm 14\%$	2/10	+3.0% (+4.0%)
medium	fine-tune only	$20 \pm 27\%$	4/7	+3.1% (+3.7%)
medium	fine-pruning	$26 \pm 26\%$	3/7	+3.1% (+3.7%)
medium	random units	$61 \pm 48\%$	2/7	+3.4% (+3.6%)
medium	purification	$79 \pm 27\%$	0/7	+3.1% (+4.0%)

Table 4: Random trigger, test split, input length 12, seeds 200 to 209, $r = 30\%$ for the methods that suppress units. Gap is the share of the gap closed, as mean $\pm$ sd over the models that were implanted (10 on the easy pair, 7 on the medium pair). Left counts the models left implanted among those implanted before. Cost is the clean-MAE change of the poisoned model, with the change for the clean twin in brackets, averaged over all 10 models of the pair.

(Table 4). Across both pairs that is 2 of 17 models for purification, 8 of 17 for random units, 13 of 17 for fine-pruning, and 14 of 17 for fine-tuning alone. Purification closes 61 to 79% of the gap, compared with 9 to 26% for fine-tuning and fine-pruning and 51 to 61% for random units. A model-by-model comparison is more informative than the standard deviations (7 to 48 points), because the methods are evaluated on the same models. Purification achieves a larger gap reduction than fine-tuning and fine-pruning on every model (10/10 and 7/7, respectively), with mean paired differences of 44 to 59 percentage points and all 95% confidence intervals above zero. Against random units, purification wins on 6 of 10 easy-pair models (+10 points, 95% interval -9 to +30) and on 4 of 7 medium-pair models (+18 points, 95% interval -5 to +50). A sign test finds no significant difference between the methods ($p = 0.75$ and 1.0). ASR@τ, the metric of BackTime and TimeGuard, gives the same ordering (Table 11 in the appendix). Purification reduces ASR@τ to 0.00 on the easy pair for both triggers, against 0.81 and 0.70 for fine-tuning alone, and to 0.38 on the random-trigger medium pair (0.82 for fine-tuning alone). On the learned-trigger medium pair, ASR@τ stays at 0.94 or above for every method, so it does not separate them there. Figure 2 shows that, on the easy pair, purification leaves fewer models implanted than random-unit suppression at 20% (4 vs. 9 of 10) and 30% (2 vs. 6). The methods tie at 10% (9 of 10) and 50% (1 of 10). On the medium pair, purification leaves 1 of 7 models implanted at 10% and none at higher budgets. Random-unit suppression leaves 3, 2, and 2 models implanted at 10, 20, and 30%, respectively, and none at 50%. At $r = 50\%$ on the easy pair, purification closes seven percentage points more of the gap than random-unit suppression. A value above 100% means the target-pattern error exceeds the reference error. This may reflect damage beyond simply removing the backdoor.

Transfer to a learned trigger. The defender continues to use randomly generated triggers. Table 5 evaluates the same purification procedure, without modification, on models poisoned with the learned trigger. Purification closes 62% of the

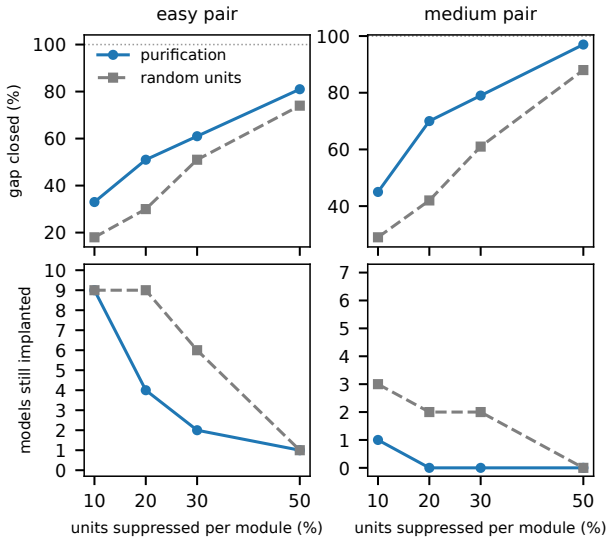


Figure 2: Top: share of the gap to a clean model closed. Bottom: number of implanted models left implanted afterwards. Both against the suppression budget, for purification and for random units (test split, input length 12, random trigger).

Pair	Method	Gap	Left	Cost (twin)
easy	fine-tune only	$14 \pm 14\%$	9/10	+2.2% (+3.7%)
easy	fine-pruning	$16 \pm 17\%$	9/10	+2.1% (+3.7%)
easy	random units	$33 \pm 26\%$	8/10	+2.4% (+3.6%)
easy	purification	$62 \pm 24\%$	2/10	+2.4% (+4.0%)
medium	fine-tune only	$19 \pm 14\%$	4/6	+2.6% (+3.7%)
medium	fine-pruning	$21 \pm 15\%$	4/6	+2.6% (+3.7%)
medium	random units	$33 \pm 15\%$	1/6	+2.9% (+3.6%)
medium	purification	$85 \pm 45\%$	0/6	+3.0% (+4.0%)

Table 5: Models poisoned with the learned trigger, test split, input length 12, seeds 200 to 209, $r = 30\%$, same columns as Table 4. Ten easy-pair models and six medium-pair models implanted.

gap on the easy pair (SD 24 percentage points) and 85% on the medium pair, compared with 14 to 21% for fine-tuning and fine-pruning and 33% for random-unit suppression on both pairs. 2 of 10 easy-pair models are left implanted, against 8 of 10 for random units, 9 of 10 for fine-tuning, and 9 of 10 for fine-pruning. Model by model, purification outperforms fine-tuning on all 10 easy-pair models and fine-pruning on 9 of 10, and it beats random units on 7 of 10 (+28 points, 95% interval +6 to +52). The number of models left implanted is lower with purification than with random-unit suppression in 6 cases and higher in none (sign test, $p = 0.031$). On the medium pair it beats random units by +51 points (interval +19 to +82) on 5 of 6 models. At $r = 50\%$ on the easy pair purification and random units tie (67% each).

Cost. On the test split the clean-MAE cost of purification is 2.4 to 3.1% for poisoned models and 4.0% for clean twins,

Units suppressed	$r = 10\%$ Gap	$r = 10\%$ Cost	$r = 30\%$ Gap	$r = 30\%$ Cost
random trigger, 17 implanted models				
defender's top units	38%	+0.8%	69%	+4.5%
oracle (changed most)	40%	+3.6%	75%	+10.6%
scored on a clean model	27%	+2.0%	62%	+9.9%
lowest-scored units	19%	+4.2%	46%	+17.6%
random units	24%	+1.5%	46%	+8.8%
learned trigger, 16 implanted models				
defender's top units	44%	+0.6%	70%	+4.5%
oracle (changed most)	35%	+2.3%	71%	+9.3%
scored on a clean model	22%	+2.0%	43%	+10.3%
lowest-scored units	19%	+5.7%	35%	+18.8%
random units	22%	+1.7%	36%	+8.7%

Table 6: Gap closed after the recovery fine-tune, and clean-MAE change before it (the damage of the suppression itself), when different units are suppressed (test split, input 12, seeds 200 to 209).

against the 3% used for selection. Most of the cost comes from the short recovery fine-tune. Fine-tuning alone costs 2.2 to 3.1% (3.7% on twins), and suppression adds 0.0 to 0.6 points. The standard deviation across seeds is 3 to 4 percentage points for every method. On the ten clean twins, which have no backdoor, the same pipeline costs 4.0% against 3.7% for fine-tuning alone, so suppression adds 0.3 points.

Sensitivity to the defender's choices. Table 13 in the appendix changes one choice at a time on the validation split (two seeds) at $r = 30\%$. All three implanted validation models are cleared in every setting. The share of the gap closed varies by up to 33 percentage points, and no setting is best on both pairs: using 12 defender-generated triggers performs better on the easy pair and worse on the medium pair, while a gentler recovery (100 steps at a lower learning rate) has the opposite pattern and costs more on the easy pair.

Unit selection. If purification works only by damaging the model, its effectiveness should depend on how much damage it causes rather than which units it removes. Table 6 compares the defender's selected units against four alternatives at the same suppression budget: the units that changed most between the poisoned model and its clean twin (an oracle), those selected by the defender's scores on a clean twin, the lowest-scored units, and random units (three draws per model). After the recovery step the defender's units close 69% of the gap at $r = 30\%$ for the random trigger, compared with 46% for random units, 46% for the lowest-scored units and 62% for units scored on a clean model. Before the recovery step they also cause the smallest increase in clean MAE (+4.5%, compared with +8.8% for random units and +9.9% to +17.6% for the others), so damage alone does not explain the difference. In paired comparisons across models, the defender's units beat random units by +23 points (95% interval +13 to +32, 14 of 17 models) and the lowest-scored units by +22 (+9 to +35), and the differences from units scored on a clean model (+6, -4 to +17) and the oracle (-6, -23 to +12) are inconclusive. For the learned trigger they beat random units by +35 points (+24 to +46, 16 of 16 models), units scored on a clean model by +27 (+15 to +41) and the lowest-scored units by +36 (+19 to +53), and the difference from the oracle is inconclusive (0, -10 to +8).The increase in clean MAE before recovery is lower for the defender's units than for every other rule, with all paired intervals below zero. The defender's ranking agrees only modestly with the oracle ranking. The Spearman correlation is 0.28, against 0.23 for the scores of a clean model, and the top 30% of the two rankings overlap by 0.41 against a chance level of 0.30. The scores therefore find units that matter for the backdoor without closely reproducing the oracle ranking.

Clean set	easy pair fine-tune	easy pair purif.	easy pair left	medium pair purif.	medium pair left
5% train (default)	$9 \pm 7\%$	$61 \pm 14\%$	2/10	$79 \pm 27\%$	0/7
5% poisoned data	$0 \pm 1\%$	$14 \pm 9\%$	10/10	$50 \pm 29\%$	1/7
5% validation	$18 \pm 10\%$	$70 \pm 30\%$	2/10	$98 \pm 28\%$	0/7
1% train	$17 \pm 13\%$	$68 \pm 27\%$	3/10	n/a	n/a
10% train	$14 \pm 8\%$	$70 \pm 22\%$	1/10	n/a	n/a

Table 7: Gap closed (mean $\pm$ SD over the implanted models) when the defender's clean set comes from a different source or has a different size. Fine-tune is fine-tuning alone on the same set. The smaller and larger sets were run on the easy pair only.

The defender's clean set. The choice of clean set can affect purification. Table 7 varies its source and size on seeds 200 to 209 at $r = 30\%$. A clean set sampled from the poisoned training array can contain the attacker's poisoned rows. On the easy pair, purification then closes 14% of the gap and leaves all 10 models implanted, while fine-tuning alone closes almost none of it. On the medium pair, purification closes 50% and leaves 1 of 7 models implanted. A set cut from the validation split closes 70% on the easy pair and 98% on the medium pair at a clean-MAE cost of 0.2 to 0.8%. The low cost may partly reflect the proximity of the validation and test periods. Changing the set size has a clearer effect on the number of models left implanted than on the share of the gap closed: 1% of the training rows closes 68% of the gap and leaves 3 of 10 models implanted, 5% closes 61% and leaves 2, and 10% closes 70% and leaves 1.

Other head counts. On the easy pair, purification closes 70 $\pm$ 21% of the gap with 4 attention heads and 63 $\pm$ 24% with 8 heads, leaving 2 and 3 of 10 models implanted, respectively. Fine-tuning alone closes 12% and 18%, with all 10 models remaining implanted in both cases. Random-unit suppression closes 46% and 45%, leaving 5 and 6 models implanted. Purification beats random-unit suppression on every model with 4 heads and on 9 of 10 models with 8 heads (paired differences of +24 [+18, +30] and +18 [+8, +30] percentage points, respectively). On the medium pair, 5 models implant with 4 heads and 6 with 8 heads. Purification closes 71% and 84% of the gap, leaving none implanted in either case, compared with 27% and 35% for fine-tuning alone, which leaves 3 models implanted in each case. The paired difference from random-unit suppression is inconclusive with 4 heads

(-4 [-23, $+15$] points), but favors purification with 8 heads (+17 [+12, +24] points).

Detection signal. Poisoned models score lower than their same-seed clean twins on the mean top-5% unit score in all 17 random-trigger pairs and all 16 learned-trigger pairs (AUROC 0.92 and 0.91, respectively, across 10 seeds). With 4 and 8 heads the direction holds in 14 of 15 and 15 of 16 pairs (AUROC 0.91 and 0.86).

Discussion

The backdoor depends on attention and MLP components across multiple blocks. No individual component group reproduces much of it when transplanted into the clean twin, and fine-pruning, which targets only MLP activity, leaves most models implanted in our experiments. These results suggest that defenses targeting only one layer or sublayer may leave substantial parts of the backdoor intact.

Attack success also varies substantially across channel pairs and input lengths. On the easy pair, the random trigger implants in 10 of 10 seeds at input length 12 and in 6 of 10 at input length 64. On the hard pair, it implants in none (Table 1). Results for one channel pair and input length may not predict attack success on others.

Random-unit suppression followed by fine-tuning is a strong baseline. At 30%, purification does not significantly outperform random-unit suppression for the random trigger. For the learned trigger, the paired bootstrap interval favors purification, but the sign test on the gap does not find a significant difference between the methods. Both methods close the same percentage of the gap at 50% suppression. Future defenses should therefore be compared against this baseline across several suppression budgets.

The source of the defender's clean set also affects purification. When the set is sampled from the poisoned training data, it may contain rows poisoned by the attacker. In our experiments, this substantially reduces purification's effect on the easy pair. Evaluations of the defense should therefore account for how the clean set is obtained.

Limitations

The experiments cover PatchTST on ETTm1, using random and learned triggers across ten seeds. For the random trigger, the easy pair implants in 10 of 10 seeds with input length and forecast horizon both set to 12, in 9 of 10 at 24, in 5 of 10 at 48, and in 2 of 10 at 96. The medium pair implants only at 12, in 7 of 10 seeds. The localization analysis requires a clean twin, which is unavailable to the defender. The defender's variants use random triggers constructed from the same attack template as the original triggers. Unlike Li and Kim (2026), who evaluate five attack methods on language models, we evaluate two trigger families on a time-series forecaster. The implantation criterion and purification settings were selected using pilot and validation data. Purification is evaluated only on models that meet the implantation criterion, and the defender is assumed to have access to a clean set.

Conclusion

In PatchTST, the backdoor depends on both attention and MLP sublayers, particularly in the later blocks. Neither group alone reproduces much of the backdoor when transplanted into the clean twin. At a 30% per-module suppression budget, the proposed scoring method leaves 2 of 17 random-trigger models and 2 of 16 learned-trigger models implanted. Random-unit suppression leaves 8 of 17 and 9 of 16, respectively. Fine-pruning leaves 13 of 17 and 13 of 16, while fine-tuning alone leaves 14 of 17 and 13 of 16. The method closes 61 to 85% of the gap to the reference model. It outperforms fine-tuning and fine-pruning on all but one model, but its advantage over random-unit suppression is statistically significant only for the learned trigger. The results also depend on the clean set: sampling it from the poisoned training data largely removes the purification's effect on the easy pair. The experiments are limited to PatchTST and the input lengths tested here. Whether the same results hold for other architectures and longer inputs remains unknown.

References

Chen, Y.; Xie, H.; Zhang, Y.; Dai, S.; and Bie, Z. 2026. Mind the Intention: Task-Aware Backdoor Attacks for Forecast-Driven Distribution Network Operations. *arXiv preprint arXiv:2606.21846*.

Dong, C.; Sun, Z.; Bai, G.; Piao, S.; Chen, W.; and Zhang, W. E. 2025. TrojanTime: Backdoor Attacks on Time Series Classification. *arXiv preprint arXiv:2502.00646*.

Huang, Y.; Zhang, M.; Wang, Z.; Li, W.; and Yang, M. 2025. Revisiting Backdoor Attacks on Time Series Classification in the Frequency Domain. In *The Web Conference*.

Jiang, Y.; Ma, X.; Erfani, S. M.; and Bailey, J. 2022. Backdoor Attacks on Time Series: A Generative Approach. *arXiv preprint arXiv:2211.07915*.

Jiang, Y.; Ma, X.; Erfani, S. M.; Li, Y.; and Bailey, J. 2024. End-to-End Anti-Backdoor Learning on Images and Time Series. *arXiv preprint arXiv:2401.03215*.

Kotowski, K.; Shendy, R.; Nalepa, J.; Kaczmarek, A.; Płudowski, D.; Wilczyński, P.; Janicki, A.; Biecek, P.; Marzetta, A.; Pande, A.; Routhu, L. C.; Srivastava, S.; and Ntagiou, E. 2026. Trojan Horse Hunt in Deep Forecasting Models: Insights from the European Space Agency Competition. *arXiv preprint arXiv:2603.20108*.

Lamparth, M.; and Reuel, A. 2024. Analyzing and Editing Inner Mechanisms of Backdoored Language Models. In *ACM Conference on Fairness, Accountability, and Transparency*.

Li, J.; and Kim, J.-E. 2026. Purifying Generative LLMs from Backdoors without Prior Knowledge or Clean Reference. In *International Conference on Learning Representations*.

Lin, X.; Liu, Z.; Fu, D.; Qiu, R.; and Tong, H. 2024. BackTime: Backdoor Attacks on Multivariate Time Series Forecasting. In *Advances in Neural Information Processing Systems*.

Liu, K.; Dolan-Gavitt, B.; and Garg, S. 2018. Fine-Pruning: Defending Against Backdooring Attacks on Deep Neural Networks. In *International Symposium on Research in Attacks, Intrusions, and Defenses*.

Liu, Z.; Liu, X.; Xu, S.; Qiao, Y.; Zhang, Y.; and Cai, X. 2026. Beyond Immediate Activation: Temporally Decoupled Backdoor Attacks on Time Series Forecasting. *arXiv preprint arXiv:2601.04247*.

Nguyen, Q. D.; Liang, S.; Li, Y.; Huo, F.; and Tao, D. 2026. TimeGuard: Channel-wise Pool Training for Backdoor Defense in Time Series Forecasting. In *International Conference on Machine Learning*.

Nie, Y.; Nguyen, N. H.; Sinthong, P.; and Kalagnanam, J. 2023. A Time Series is Worth 64 Words: Long-term Forecasting with Transformers. In *International Conference on Learning Representations*.

Wu, D.; and Wang, Y. 2021. Adversarial Neuron Pruning Purifies Backdoored Deep Models. In *Advances in Neural Information Processing Systems*.

Xiang, K.; Yang, H.; Jiang, W.; Wang, H.; Huang, S.; Li, S.; Liu, Y.; and Guo, J. 2025. BadTime: An Effective Backdoor Attack on Multivariate Long-Term Time Series Forecasting. *arXiv preprint arXiv:2508.04189*.

Zhou, H.; Zhang, S.; Peng, J.; Zhang, S.; Li, J.; Xiong, H.; and Zhang, W. 2021. Informer: Beyond Efficient Transformer for Long Sequence Time-Series Forecasting. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

Appendix: Additional Results

The tables below give the budget sweeps, the raw-unit and attack-success results, the ablation values, the validation sweep and the pilot settings referred to in the main text.

Method	$r = 10\%$	20%
random, easy (10 models)		
fine-tune only	9±7 (10/10), no suppression	
fine-pruning	10±8 (10/10)	13±10 (10/10)
random units	18±16 (9/10)	30±14 (9/10)
purification	33±13 (9/10)	51±18 (4/10)
random, medium (7 models)		
fine-tune only	20±27 (4/7), no suppression	
fine-pruning	21±27 (4/7)	23±27 (3/7)
random units	29±22 (3/7)	42±29 (2/7)
purification	45±20 (1/7)	70±25 (0/7)
learned, easy (10 models)		
fine-tune only	14±14 (9/10), no suppression	
fine-pruning	14±15 (9/10)	15±16 (9/10)
random units	16±9 (10/10)	31±13 (9/10)
purification	41±18 (5/10)	57±19 (2/10)
learned, medium (6 models)		
fine-tune only	19±14 (4/6), no suppression	
fine-pruning	19±14 (4/6)	20±15 (3/6)
random units	28±12 (0/6)	45±39 (1/6)
purification	48±41 (1/6)	71±37 (0/6)

Table 8: Gap closed (%, mean ± sd across initially implanted models), followed in brackets by the number of models remaining implanted out of the initially implanted models (test split, input length 12, seeds 200 to 209). Block titles show the fine-tuning-only results, which do not use a suppression budget. Results are shown for suppression budgets of 10% and 20%.

Method	$r = 30\%$	50%
random, easy (10 models)		
fine-tune only	9±7 (10/10), no suppression	
fine-pruning	17±10 (10/10)	25±11 (9/10)
random units	51±28 (6/10)	74±23 (1/10)
purification	61±14 (2/10)	81±20 (1/10)
random, medium (7 models)		
fine-tune only	20±27 (4/7), no suppression	
fine-pruning	26±26 (3/7)	37±26 (3/7)
random units	61±48 (2/7)	88±37 (0/7)
purification	79±27 (0/7)	97±36 (0/7)
learned, easy (10 models)		
fine-tune only	14±14 (9/10), no suppression	
fine-pruning	16±17 (9/10)	21±20 (9/10)
random units	33±26 (8/10)	67±34 (3/10)
purification	62±24 (2/10)	67±23 (2/10)
learned, medium (6 models)		
fine-tune only	19±14 (4/6), no suppression	
fine-pruning	21±15 (4/6)	26±16 (1/6)
random units	33±15 (1/6)	107±68 (0/6)
purification	85±45 (0/6)	122±46 (0/6)

Table 9: Gap closed (%, mean ± sd across initially implanted models), followed in brackets by the number of models remaining implanted out of those initially implanted (test split, input length 12, seeds 200 to 209). Block titles show the fine-tuning-only results, which have no suppression budget. Results are shown for suppression budgets of 30% and 50%.

Trigger	Pair	Poisoned	Twin	Flat	ρ raw	ρ std.
random	easy	0.103	1.552	1.593	0.08	0.07
random	medium	0.585	1.587	1.593	0.45	0.47
learned	easy	0.113	1.230	1.593	0.10	0.08
learned	medium	0.335	0.714	1.593	0.48	0.47

Table 10: MAE_P in raw units for the poisoned models, their clean twins and the flat forecaster (seed means, seeds 200 to 209, input length 12), and the criterion's ratio ρ computed in raw and in standardized units.

Trigger	Pair	fine-tune	fine-prune	random	purif.
random	easy	0.81	0.54	0.05	0.00
random	medium	0.82	0.81	0.45	0.38
learned	easy	0.70	0.67	0.24	0.00
learned	medium	1.00	1.00	1.00	0.94

Table 11: ASR@τ after each method at $r = 30\%$ (test split, seeds 200 to 209, implanted models), where τ is the model's own clean MAE and ASR is the share of triggered windows whose MAE_P is below it. Lower is better.

Reverted to twin	easy pair	medium pair
all attention	0.13	0.19
all MLP	0.33	0.33
attention and MLP	-0.07	-0.02
embedding	0.62	0.82
head	0.49	0.86
normalization	0.79	0.91
matched-norm noise, attention	0.78	0.94
other clean model, attention	-0.17	-0.14

Table 12: Retained share of the backdoor for each group (input length 12, implanted seeds only: 10 easy, 7 medium).

	easy		medium	
Setting	Gap	Cost	Gap	Cost
default	61%	+2.7%	52%	+1.8%
12 variants	67%	+3.2%	38%	+1.8%
stronger defender trigger	63%	+2.7%	44%	+1.7%
longer variant fine-tune	62%	+2.6%	50%	+1.6%
gentler recovery	51%	+4.9%	71%	+2.2%
re-initialize, not zero	62%	+2.7%	52%	+1.8%

Table 13: Purification at $r = 30\%$ on the validation split (two seeds) with one change at a time. Gap is the share of the gap closed over the models that implanted (two on the easy pair, one on the medium pair), and cost is the clean-MAE change of the poisoned model.